

Claude e Claude Code in pratica

Dal prompting agli agenti

Simone Scannapieco

Corso base per Tecnodata Trentina S.r.l., Trento, Italia

Modulo 5 — Skills

Anatomia di uno SKILL.md

Due parti: frontmatter e corpo

Che cosa c'è in un file SKILL.md, e i campi di questo modulo

- Una skill è una cartella con un file SKILL.md, in **due parti**: il frontmatter YAML, fra due righe ---, che dice a Claude quando usarla; sotto, il Markdown con le istruzioni
- In Claude Code i campi sono venti: «All fields are optional. Only description is recommended». Lo standard Agent Skills chiede name e description. Il corso li scrive **sempre tutti e due**

il campo	che cosa fa
name	per lo standard, uguale alla cartella, fino a 64 caratteri, minuscole, cifre e trattini
description	che cosa fa e quando usarla; per lo standard, fino a 1.024 caratteri
when_to_use, argument-hint	frasi di trigger in più; gli argomenti attesi
disable-model-invocation, user-invocable	chi la invoca: la slide «Chi la invoca»
allowed-tools, disallowed-tools	strumenti senza chiedere; strumenti tolti
context, compatibility	fork, in un subagent; i requisiti d'ambiente

- Un campo scritto male si ignora, **senza errori**. Il --- di apertura va sulla **prima riga**. Con un frontmatter malformato /nome funziona ancora, ma Claude non può confrontare la richiesta con la description

Il trigger sta nella `description`

Nessun campo si chiama trigger: la `description` dice che cosa fa, e quando

- `description` dice che cosa fa la skill e quando usarla: «Claude uses this to decide when to apply the skill.» Per un articolo di Anthropic, «the name and description are the only parts of the SKILL.md file that influence triggering»; la pagina delle skill aggiunge `when_to_use`, che si accoda alla `description`; per la guida dello standard, «the description carries the entire burden of triggering»
- «This isn't keyword matching—Claude understands semantic relationships. However, vague descriptions reduce triggering accuracy.»

da evitare: dall'elenco «Avoid» di Anthropic

```
description: Does stuff with files
```

da fare: la demo che segue, a capo per la slide

```
description: Controlla la configurazione di un apparato di rete
  contro la checklist di audit del team. Da usare quando si chiede
  di controllare o rivedere la configurazione di un apparato.
```

- Dalle best practice di Anthropic: «Include both what the Skill does and specific triggers/contexts for when to use it.» «Always write in third person.» E parole chiave specifiche, chiede lo standard

Il trigger sta nella `description`

Nessun campo si chiama trigger: la `description` dice che cosa fa, e quando

- Né troppo larga né troppo stretta: «Overly generic descriptions cause inappropriate activation, while missing use cases cause missed activations.» «too broad and you get false triggers, too narrow and it never fires»
- Le parole di chi chiede: «Check the description includes keywords users would naturally say». Per lo standard, «Focus on user intent, not implementation.»
- E non basta sempre. Ancora lo standard: «A simple, one-step request like “read this PDF” may not trigger a PDF skill even if the description matches perfectly»
- La skill `skill-creator` di Anthropic: «currently Claude has a tendency to “undertrigger” skills», e chiede description «a little bit “pushy”»
- Nell’elenco, `description` e `when_to_use` si troncano a 1.536 caratteri: «Put the key use case first»

La `description` non è un riassunto: dice quando.

«the description field is not a summary, it’s a description of when to trigger this skill.»

M5-D02 La descrizione decide

Demo del docente in Claude Code: cambia soltanto la `description`

- Da «How to create Skills for Claude: steps and examples»: «vague descriptions reduce triggering accuracy». Un **confronto**
- Dalla pagina delle skill: «disable-model-invocation: true: Claude can't invoke the skill on its own». Dalla pagina di `claude -p`: «Include /skill-name in the prompt string and Claude Code expands it before running.» Due **garanzie**
- Lo stesso corpo, con un **nome neutro**, e la stessa richiesta: «Controlla la configurazione di sw-prova-01 (configurazione.txt): ci sono problemi?»

braccio	che cosa cambia
vaga	`description: Aiuta con i file`
specifica	la `description` di prima, «Controlla la configurazione …»
near-miss	la specifica; la richiesta: «Quante righe ha il file configurazione.txt?»
con il flag	la specifica, più `disable-model-invocation: true`
`/prova-d02`	la stessa con il flag; il nome in testa alla richiesta

- Una cartella nuova e una sessione nuova per chiamata, Sonnet, effort low, i soli strumenti Read e Skill. Guardate se nello stream c'è una chiamata a `Skill`

```
m5_skills/s4_skills/m5_4_anatomia/M5-D02_la_descrizione_decide/\
demo/in_classe.sh ~/corso-claude-code/M5-D02
```

- **Indovinate, a voce**: con la descrizione "Aiuta con i file", quante volte su tre Claude userà la skill?

M5-D02 La descrizione decide

Demo del docente in Claude Code: cambia soltanto la `description`

braccio	chiamate a `Skill`	il marcatore `AUD-01` nella risposta
vaga	**0 su 3**	0 su 3
specifica	**3 su 3**	3 su 3
near-miss	0 su 3	0 su 3
con il flag	0 su 3	0 su 3
`/prova-d02` con il flag		3 su 3

Tre esecuzioni, ogni chiamata una sessione nuova: i valori che il controllo ha stampato, in tabella
Registrazione del corso, 3 novembre 2026, Claude Code 2.1.317, Sonnet, effort low

- La risposta: **0 su 3**, con un nome neutro. «Specifica contro vaga, «vague descriptions reduce triggering accuracy»: **confermata** (3 su 3 contro 0 su 3)»
- «Con il flag, «Claude can't invoke the skill on its own»: **confermata 3 su 3**»; «/prova-d02 con il flag, «Include /skill-name in the prompt string»: **confermata 3 su 3**»
- «Near-miss, come dato: Skill 0 su 3; una near-miss non dovrebbe far scattare la skill»
- Nello stream, una chiamata è un `tool_use` di nome `Skill`, e il suo risultato dice «Launching skill»; con `/nome` in testa non c'è. Come dato, la prima richiesta: specifica 7.192 token, con il flag 7.128; per la documentazione, il flag tiene la description fuori dal contesto
- La vaga è un esempio cattivo, apposta: `tools/controlla_skill.py` la segnala prima di ogni prova, «AVVISO description: non dice quando usarla»

Le istruzioni

Il corpo di uno `SKILL.md`: sezioni Markdown, ciò che a Claude manca

- Che cosa scrivere, dalle best practice di Anthropic: «Only add context Claude doesn't already have.» La domanda dello standard, per ogni riga: «Would the agent get this wrong without this instruction?»
- Conciso, dalla pagina delle skill: «every line is a recurring token cost». Per lo standard, «Concise, stepwise guidance with a working example tends to outperform exhaustive documentation.»
- Quanto specifico: «Match the level of specificity to the task's fragility and variability.»
- Quanto lungo: «Keep SKILL.md under 500 lines. Move detailed reference material to separate files.» E per lo standard, sotto i 5.000 token raccomandati
- Per tutto il compito: il corpo entra come un unico messaggio, e «Claude Code does not re-read the skill file on later turns». Quindi «Run the tests after every edit», non «Run the tests»
- Il più importante in cima: «Truncation keeps the start of the file»

Le istruzioni

Il corpo di uno `SKILL.md`: sezioni Markdown, ciò che a Claude manca

- Sezioni Markdown: lo standard raccomanda i passi, esempi di input e output, i casi limite. E un metodo, non una risposta: «how to approach a class of problems, not what to produce for a specific instance»
- Per i flussi complessi, una **checklist** «that Claude can copy into its response and check off as it progresses»
- Un **modello** del formato: «Provide templates for output format.» Per lo standard, «more reliable than describing the format in prose»
- Un **validatore**: «Run validator → fix errors → repeat»
- I **gotcha**, dal team di Claude Code: «The highest-signal content in any skill is the Gotchas section.» Per lo standard, «a list of gotchas» è spesso il contenuto di maggior valore, e «When an agent makes a mistake you have to correct, add the correction to the gotchas section.»
- E, dalle best practice di Anthropic: «No time-sensitive information», «Consistent terminology throughout», «Examples are concrete, not abstract»

Gli esempi

Una sezione del corpo, o un file di supporto: coppie di input e output

- Nessun campo per gli esempi: una **sezione Markdown** del corpo, o un file di supporto
- «provide input/output pairs just like in regular prompting»: trasmettono lo stile voluto «more clearly than descriptions alone». Concreti; per lo standard, «Examples of inputs and outputs»

`revisione-pr`, a capo per la slide: «[…]» segna un taglio

````
## Esempi

Una riga aggiunta in un file nuovo di `src/Report/`:

```php
public function totale($cliente)
```

La sua riga nella revisione:

```markdown
| 4.1 | src/Report/ReportFatturato.php | 12 | parametro e valore
di ritorno senza tipo | `int $clienteId`, ritorno `int` |
```
[…]
````

File di supporto e script

Oltre il file: si citano dicendo quando caricarli; uno script si esegue

- Per lo standard: `references/`, `scripts/`, `assets/`; percorsi relativi alla radice della skill, «one level deep». Per le best practice di Anthropic, le barre in avanti, «even on Windows»
- Citateli dallo `SKILL.md`: «so Claude knows what each file contains and when to load it»; per lo standard, «The key is telling the agent when to load each file.» Così un documento grande non si carica a ogni esecuzione
- Uno script: dite se va **eseguito** o letto. Eseguito, «the script code itself never enters context», e Claude riceve solo l'output; ed è «more reliable than generated code»
- Per lo standard: niente richieste interattive, un `--help`, formati strutturati, i dati su stdout e i messaggi su stderr
- `${CLAUDE_SKILL_DIR}` è la cartella dello `SKILL.md`, da qualunque cartella di lavoro; `${CLAUDE_PROJECT_DIR}` la radice del progetto. Un dato, da una prova del corso del 3 novembre 2026, Claude Code 2.1.317: un file nominato senza percorso è stato cercato prima nella cartella della skill

Chi la invoca

Voi, Claude, o tutti e due; e dove si scrive il nome

nel frontmatter	voi	Claude	nelle parole della pagina
(di serie)	sì	sì	«Description always in context, full skill loads when invoked»
`disable-model-invocation: true`	sì	non da solo	«Description not in context, full skill loads when invoked»
`user-invocable: false`	no	sì	«hides it from the / menu and doesn't run it when you type /name»

Dalla pagina delle skill di Claude Code

- Per eseguirla voi: «put its name at the start of your message. After plain text, the name gives Claude permission to run the skill but doesn't run it»
- Un **dato** da una prova del corso del 3 novembre 2026, non una regola della documentazione: con `claude -p` in dontAsk, dove ciò che chiederebbe un permesso è negato, Claude Code 2.1.317, una skill con `allowed-tools` o `disallowed-tools` che Claude invoca da sé è negata, finché le regole non nominano `Skill(nome)`